Why NumPy?

NumPy is a powerful library in Python that makes numerical computations easy and efficient.

1. NumPy provides efficient storage.
2. It also provides better ways of handling data for processing.
3. It is fast.
4. It is easy to learn.
5. NumPy uses relatively less memory to store data.

Importing the NumPy library

In [1]:
```
import numpy as np
```

In [2]:
```
list1=[1,2,3,4,5]
print(list1)
type(list1)
```

```
[1, 2, 3, 4, 5]
```

Out[2]:
```
list
```

In [3]:
```
#Converting list to NumPy array
arr = np.array(list1)
print(arr)
type(arr)
```

```
[1 2 3 4 5]
```

Out[3]:
```
numpy.ndarray
```

In [4]:
```
listarray= np.array([[1,2,3],[5,5,8],[4,6,9]])
print(listarray)
listarray.size #Number of elements in an array
```

```
[[1 2 3]
 [5 5 8]
 [4 6 9]]
```

Out[4]:
```
9
```

In [5]:
```
listarray.dtype #Checking the data type of the values in the array
```

Out[5]:
```
dtype('int32')
```

Creating a simple 1D array

In [6]:
```
arr = np.array([1, 2, 3, 4, 5])
print("1D Array:")
print(arr)
arr.shape
```

```
1D Array:
[1 2 3 4 5]
```

Out[6]:
```
(5,)
```

Creating a 2D array (Matrix)

In [7]:
```
arr_2d= np.array([[1, 2, 3], [4, 5, 6]])
print("2D Array:")
print(arr_2d)
arr_2d.shape #the number of rows and columns an array
```

```
2D Array:
[[1 2 3]
 [4 5 6]]
```

Out[7]:
```
(2, 3)
```

In [8]:
```
a=np.array([[1,2,3,2],[2,5,82,7],[5,8,7,5]],dtype=np.int32)
print(a)
a.shape
#Number of rows (axis 0): 3(Refers to the vertical direction in the array)
#Number of columns (axis 1): 4(Refers to the horizontal direction in the array,
```

```
[[ 1  2  3  2]
 [ 2  5 82  7]
 [ 5  8  7  5]]
```

Out[8]:
```
(3, 4)
```

Creating a 3D array

In [9]:
```
# Create a 3D array with dimensions (2, 3, 4)
# 2 matrices(layers), 3 rows, and 4 columns
arr_3d = np.array([[[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]],
                   [[13, 14, 15, 16], [17, 18, 19, 20], [21, 22, 23, 24]]],
                   dtype=float)

print(arr_3d)
print("Shape:", arr_3d.shape)
```

```
[[[ 1.  2.  3.  4.]
  [ 5.  6.  7.  8.]
  [ 9. 10. 11. 12.]]

 [[13. 14. 15. 16.]
  [17. 18. 19. 20.]
  [21. 22. 23. 24.]]]
Shape: (2, 3, 4)
```

In [10]:
```
arr_3d.ndim #Number of Dimensions
```

Out[10]:
```
3
```

In [11]:
```
zeros_3d = np.zeros((2, 3, 4))
zeros_3d
```

Out[11]:
```
array([[[0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]],

       [[0., 0., 0., 0.],
        [0., 0., 0., 0.],
        [0., 0., 0., 0.]]])
```

In [12]:
```
ones_3d = np.ones((2, 3, 4))
ones_3d
```

Out[12]:
```
array([[[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]],

       [[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]]])
```

In [13]:
```
random_3d = np.random.random((2, 3, 4))
random_3d
```

Out[13]:
```
array([[[0.98835175, 0.69182741, 0.39687112, 0.03449365],
        [0.59837054, 0.93019528, 0.26655287, 0.35428686],
        [0.02402669, 0.89223808, 0.89911346, 0.18816227]],

       [[0.85583822, 0.23225966, 0.71509715, 0.53106299],
        [0.95348822, 0.25764291, 0.59329536, 0.08893972],
        [0.03570523, 0.80498216, 0.09447798, 0.11753341]]])
```

Creating a zero-filled array

In [14]:
```
zeros = np.zeros((2, 3))  #2x3 array of zeros
print("Zero-filled Array:")
print(zeros)
```

```
Zero-filled Array:
[[0. 0. 0.]
 [0. 0. 0.]]
```

In [15]:
```
zeros.dtype
```

Out[15]:
```
dtype('float64')
```

In [16]:
```
zeros.shape
```

Out[16]:
```
(2, 3)
```

Creating an array of ones

In [17]:
```
ones = np.ones((3, 3))  #3x3 array of ones
print("Ones-filled Array:")
print(ones)
```

```
Ones-filled Array:
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

Creating an array of a Particlular value

In [18]:
```
array_of_threes= np.full((5,4),3) #5x4 array filled with 3
print(array_of_threes)
```

```
[[3 3 3 3]
 [3 3 3 3]
 [3 3 3 3]
 [3 3 3 3]
 [3 3 3 3]]
```

In [19]:
```
# Create a 1D array of length 5, filled with 2 and with dtype float64
array_of_twos_float = np.full(5, 2, dtype=np.float64)
print(array_of_twos_float)
```

```
[2. 2. 2. 2. 2.]
```

In [20]:
```
# Creating an empty array of shape (4, 6)
emp = np.empty((4, 6)) #Creates a NumPy array without initializing its values
emp
```

Out[20]:
```
array([[0.98835175, 0.69182741, 0.39687112, 0.03449365, 0.59837054,
        0.93019528],
       [0.26655287, 0.35428686, 0.02402669, 0.89223808, 0.89911346,
        0.18816227],
       [0.85583822, 0.23225966, 0.71509715, 0.53106299, 0.95348822,
        0.25764291],
       [0.59329536, 0.08893972, 0.03570523, 0.80498216, 0.09447798,
        0.11753341]])
```

Identity matrix

In [21]:
```
# Create a 45x45 identity matrix

ide = np.identity(45)
#the diagonal elements are set to 1, with all other elements being 0.

print(ide)
ide.shape
```

```
[[1. 0. 0. ... 0. 0. 0.]
 [0. 1. 0. ... 0. 0. 0.]
 [0. 0. 1. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 1. 0. 0.]
 [0. 0. 0. ... 0. 1. 0.]
 [0. 0. 0. ... 0. 0. 1.]]
```

Out[21]:
```
(45, 45)
```

In [22]:
```
ide.size
```

Out[22]:
```
2025
```

In [23]:
```
ide.dtype
```

Out[23]:
```
dtype('float64')
```

In [24]:
```
# Create an 8x8 identity matrix

eye_matrix = np.eye(8)
#you can specify the number of rows and columns or shift the diagonal.

print(eye_matrix)
```

```
[[1. 0. 0. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1. 0. 0.]
 [0. 0. 0. 0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 0. 0. 0. 1.]]
```

Create a numpy array with random values

In [25]:
```
# Create a 3x4 array with random float values between 0 and 1
b=np.random.random((3,4))
b
```

Out[25]:
```
array([[0.61654747, 0.123858  , 0.74552042, 0.18347791],
       [0.29773944, 0.95492384, 0.52970465, 0.94309887],
       [0.36198761, 0.98647445, 0.78015207, 0.60135261]])
```

Create a random integer values array within a specific range

In [26]:
```
#Create a 3x5 array with random integers between 10(inclusive) and 100(exclusi
random_integers = np.random.randint(10, 100, (3, 5))

print(random_integers)
```

```
[[33 95 32 42 84]
 [11 81 47 33 72]
 [90 50 87 53 48]]
```

In [27]:
```
#creates a sequence with a specified number of points in a specified range
lspace=np.linspace(1,10,6)
# Creating an array with 6 equally(evenly) spaced values between 1 and 10
lspace
```

Out[27]:
```
array([ 1. ,  2.8,  4.6,  6.4,  8.2, 10. ])
```

In [28]:
```
rng=np.arange(15)
rng
```

Out[28]:
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])
```

In [29]:
```
#creates a sequence with a specified step size
#but doesn't guarantee the no. of elements
arr_arange = np.arange(1, 10, 2) #1 to 10 with step size of 2
arr_arange
```

Out[29]:
```
array([1, 3, 5, 7, 9])
```

In [30]:
```
rng = np.arange(10, 45) # Creating an array from 10 to 45

print(rng)
```

```
[10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
 34 35 36 37 38 39 40 41 42 43 44]
```

In [31]:
```
arr=np.arange(99) #Create a 1D array with 99 values from 0 to 98
arr
```

Out[31]:
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
       34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50,
       51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67,
       68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84,
       85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98])
```

In [32]:
```
arr.reshape(3,33)
#Reshaping the 1D array into a 3x33 2D array(3 rows and 33 columns)
```

Out[32]:
```
array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15,
        16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31,
        32],
       [33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48,
        49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64,
        65],
       [66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81,
        82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97,
        98]])
```

In [33]:
```
arr=arr.ravel() #This flattens the reshaped 2D array back into a 1D array
arr
```

Out[33]:
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
       34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50,
       51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67,
       68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84,
       85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98])
```

In [34]:
```
arr.shape
```

Out[34]:
```
(99,)
```

Mathematical operations on a np array and Array Manipulation

In [35]:
```
x=[[1,2,3],[4,5,6],[7,1,0]]
ar=np.array(x)
ar
```

Out[35]:
```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 1, 0]])
```

In [73]:
```
ar.sum(axis=1) # Sum along axis 1 (sum of each row)
#means the operation works across columns (horizontally) for each row.
```

Out[73]:
```
array([ 6, 15,  8])
```

In [74]:
```
ar.sum(axis=0) # Sum along axis 0 (sum of each column)
#means the operation works across rows (vertically) for each column.
```

Out[74]:
```
array([12,  8,  9])
```

In [38]:
```
ar.T #Transpose of the array
```

Out[38]:
```
array([[1, 4, 7],
       [2, 5, 1],
       [3, 6, 0]])
```

In [39]:
```
ar.nbytes #checking the no. of bytes(memory) consumption of an array
```

Out[39]:
```
36
```

In [40]:
```
one=np.array([1,2,5,4,95,8])
one.argmax()  #Index of the maximum value in one
```

Out[40]:
```
4
```

In [41]:
```
one.argmin()  #Index of the minimum value in one
```

Out[41]:
```
0
```

In [42]:
```
one.argsort() #Indices that would sort the array
```

Out[42]:
```
array([0, 1, 3, 2, 5, 4], dtype=int64)
```

In [43]:
```
ar
```

Out[43]:
```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 1, 0]])
```

In [44]:
```
ar.argmin() #index of the minimum value in the entire array
```

Out[44]:
```
8
```

In [45]:
```
ar.argmax()  #index of the maximum value in the entire array
```

Out[45]:
```
6
```

In [46]:
```
ar.argsort()  #indices that would sort the array
```

Out[46]:
```
array([[0, 1, 2],
       [0, 1, 2],
       [2, 1, 0]], dtype=int64)
```

In [47]:
```
ar.argmax(axis=1) #Index of the maximum value in each row
```

Out[47]:
```
array([2, 2, 0], dtype=int64)
```

In [76]:
```
ar.argmin(axis=1) #Index of the minimum value in each row
```

Out[76]:
```
array([0, 0, 2], dtype=int64)
```

In [77]:
```
ar.argmax(axis=0) #Index of the maximum value in each column
```

Out[77]:
```
array([2, 1, 1], dtype=int64)
```

In [78]:
```
ar.argmin(axis=0) #Index of the minimum value in each column
```

Out[78]:
```
array([0, 2, 2], dtype=int64)
```

In [49]:
```
ar.ravel()
```

Out[49]:
```
array([1, 2, 3, 4, 5, 6, 7, 1, 0])
```

In [50]:
```
ar2=np.array([[1,2,3],[2,5,4],[7,8,9]])
ar2
```

Out[50]:
```
array([[1, 2, 3],
       [2, 5, 4],
       [7, 8, 9]])
```

In [51]:
```
ar
```

Out[51]:
```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 1, 0]])
```

In [52]:
```
ar+ar2  #Adding two arrays
```

Out[52]:
```
array([[ 2,  4,  6],
       [ 6, 10, 10],
       [14,  9,  9]])
```

In [53]:
```
[324,32]+[34,545]
```

Out[53]:
```
[324, 32, 34, 545]
```

In [54]:
```
ar*ar2 #Element-wise multiplication
```

Out[54]:
```
array([[ 1,  4,  9],
       [ 8, 25, 24],
       [49,  8,  0]])
```

In [55]:
```
np.square(ar) #Element-wise square of the array
```

Out[55]:
```
array([[ 1,  4,  9],
       [16, 25, 36],
       [49,  1,  0]])
```

In [56]:
```
np.sqrt(ar2) #Element-wise square root of the array
```

Out[56]:
```
array([[1.        , 1.41421356, 1.73205081],
       [1.41421356, 2.23606798, 2.        ],
       [2.64575131, 2.82842712, 3.        ]])
```

In [57]:
```
ar.sum() #Sum of all elements in the array
```

Out[57]:
```
29
```

In [58]:
```
ar2.min() #Minimum value in the entire array
```

Out[58]:
```
1
```

In [59]:
```
ar.max()  #Maximum value in the entire array
```

Out[59]:
```
7
```

In [60]:
```
list1 = [1,2,3,4,5]
list2 = [6,7,8,9,10]

print(list1 + list2)    #concatenate or joins two list
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

In [61]:
```
a = np.random.randint(0,10,(3,3))
b = np.random.randint(10,20,(3,3))
```

In [62]:
```
print(a)
print(b)
```

```
[[2 9 6]
 [5 6 8]
 [1 1 7]]
[[15 16 17]
 [12 19 16]
 [18 15 11]]
```

In [63]:
```
print(a+b)
print(a-b)
print(a*b)
print(a/b)
```

```
[[17 25 23]
 [17 25 24]
 [19 16 18]]
[[-13  -7 -11]
 [ -7 -13  -8]
 [-17 -14  -4]]
[[ 30 144 102]
 [ 60 114 128]
 [ 18  15  77]]
[[0.13333333 0.5625     0.35294118]
 [0.41666667 0.31578947 0.5       ]
 [0.05555556 0.06666667 0.63636364]]
```

In [64]:
```
a = np.random.randint(0,10,(3,3))
b = np.random.randint(10,20,(3,3))
```

In [65]:
```
print(a)
print(b)
```

```
[[4 8 5]
 [1 2 9]
 [4 4 5]]
[[12 14 11]
 [14 14 15]
 [15 12 17]]
```

In [66]:
```
print(np.add(a,b))
print(np.subtract(a,b))
print(np.multiply(a,b))
print(np.divide(a,b))
```

```
[[16 22 16]
 [15 16 24]
 [19 16 22]]
[[ -8  -6  -6]
 [-13 -12  -6]
 [-11  -8 -12]]
[[ 48 112  55]
 [ 14  28 135]
 [ 60  48  85]]
[[0.33333333 0.57142857 0.45454545]
 [0.07142857 0.14285714 0.6       ]
 [0.26666667 0.33333333 0.29411765]]
```

In [67]:
```
array = np.random.randint(0,10,(2,3))
print(array)
print(array.shape)
```

```
[[3 3 3]
 [6 4 1]]
(2, 3)
```

In [68]:
```
trans = np.transpose(array)
print(trans)
print(trans.shape)
```

```
[[3 6]
 [3 4]
 [3 1]]
(3, 2)
```

In [69]:
```
array = np.random.randint(0,10,(2,3))
print(array)
print(array.shape)
```

```
[[4 1 1]
 [6 4 4]]
(2, 3)
```

In [70]:
```
trans2 = array.T
print(trans2)
print(trans2.shape)
```

```
[[4 6]
 [1 4]
 [1 4]]
(3, 2)
```

In [71]:
```
a = np.random.randint(0,10,(2,3))
print(a)
print(a.shape)
```

```
[[7 9 6]
 [2 2 3]]
(2, 3)
```

In [72]:
```
b = a.reshape(3,2)
print(b)
print(b.shape)
```

```
[[7 9]
 [6 2]
 [2 3]]
(3, 2)
```